

Consider the example in the previous section. The disk-bound process needed only 10% of the processor to keep busy, but Round Robin only gave it 0.5% of the processor, while each of the two compute-bound processes received nearly 50%. Max-min fairness would assign 10% of the processor to the I/O-bound process, and it would split the remainder equally between the two compute-bound processes, with 45% each.

A hypothetical but completely impractical implementation of max-min would be to give the processor at each instant to whichever process has received the least portion of the processor. In the example above, the disk-bound task would always be scheduled instantly, preempting the compute-bound processes. However, we have already seen why this would not work well. With two equally long tasks, as soon as we execute one instruction in one task, it would have received more resources than the other one, so to preserve “fairness” we would need to instantly switch to the next task.

We can approximate a max-min fair allocation by relaxing this constraint — to allow a process to get ahead of its fair allocation by one time quantum. Every time the scheduler needs to make a choice, it chooses the task for the process with the least accumulated time on the processor. If a new process arrives on the queue with much less accumulated time, such as the disk-bound task, it will preempt the process, but otherwise the current process will complete its quantum. Tasks may get up to one time quantum more than their fair share, but over the long term the allocation will even out.

The algorithm we just described was originally defined for network, and not processor, scheduling. If we share a link between a browser request and a long download, we will get reasonable responsiveness for the browser if we have approximately fair allocation — the browser needs few network packets, and so under max-min its packets will always be scheduled ahead of the packets from the download.

Even this approximation, though, can be computationally expensive, since it requires tasks to be maintained on a priority queue. For some server environments, there can be tens or even hundreds of thousands of scheduling decisions to be made every second. To reduce the computational overhead of the scheduler, most commercial operating systems use a somewhat different algorithm, to the same goal, which we describe next.

7.1.5 Case Study: Multi-Level Feedback

Most commercial operating systems, including Windows, MacOS, and Linux, use a scheduling algorithm called [multi-level feedback queue \(MFQ\)](#). MFQ is designed to achieve several simultaneous goals:

- **Responsiveness.** Run short tasks quickly, as in SJF.
- **Low Overhead.** Minimize the number of preemptions, as in FIFO, and minimize the time spent making scheduling decisions.
- **Starvation-Freedom.** All tasks should make progress, as in Round Robin.
- **Background Tasks.** Defer system maintenance tasks, such as disk defragmentation, so they do not interfere with user work.

- **Fairness.** Assign (non-background) processes approximately their max-min fair share of the processor.

As with any real system that must balance several conflicting goals, MFQ does not perfectly achieve any of these goals. Rather, it is intended to be a reasonable compromise in most real-world cases.

MFQ is an extension of Round Robin. Instead of only a single queue, MFQ has multiple Round Robin queues, each with a different priority level and time quantum. Tasks at a higher priority level preempt lower priority tasks, while tasks at the same level are scheduled in Round Robin fashion. Further, higher priority levels have *shorter* time quanta than lower levels.

Tasks are moved between priority levels to favor short tasks over long ones. A new task enters at the top priority level. Every time the task uses up its time quantum, it drops a level; every time the task yields the processor because it is waiting on I/O, it stays at the same level (or is bumped up a level); and if the task completes it leaves the system.

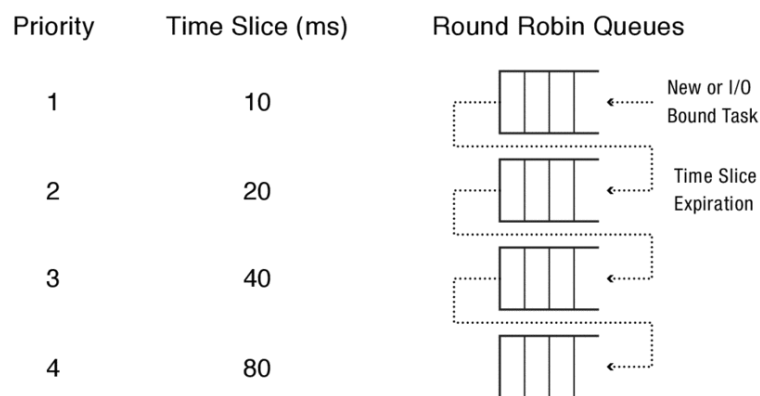


Figure 7.5: Multi-level Feedback Queue when running a mixture of I/O-bound and compute-bound tasks. New tasks enter at high priority with a short quantum; tasks that use their quantum are reduced in priority.

Figure 7.5 illustrates the operation of an MFQ with four levels. A new compute-bound task will start as high priority, but it will quickly exhaust its time quantum and fall to the next lower priority, and then the next. Thus, an I/O-bound task needing only a modest amount of computing will almost always be scheduled quickly, keeping the disk busy. Compute-bound tasks run with a long time quantum to minimize switching overhead while still sharing the processor.

So far, the algorithm we have described does not achieve starvation freedom or max-min fairness. If there are too many I/O-bound tasks, the compute-bound tasks may receive no time on the processor. To combat this, the MFQ scheduler monitors every process to ensure it is receiving its fair share of the resources. At each level, Linux actually maintains two queues — tasks whose processes have already reached their fair share are only scheduled if all other processes at that level have also received their fair share.

Periodically, any process receiving less than its fair share will have its tasks increased in priority; equally, tasks that receive more than their fair share can be reduced in priority.

Adjusting priority also addresses strategic behavior. From a purely selfish point of view, a task can attempt to keep its priority high by doing a short I/O request immediately before its time quantum expires. Eventually the system will detect this and reduce its priority to its fair-share level.

Our previously hapless supermarket manager reads a bit farther into the textbook and realizes that supermarket express lanes are a form of multi-level queue. By limiting express lanes to customers with a few items, the manager can ensure short tasks complete quickly, reducing average response time. The manager can also monitor wait times, adding extra lanes to ensure that everyone is served reasonably quickly.

7.1.6 Summary

We summarize the lessons from this section:

- FIFO is simple and minimizes overhead.
- If tasks are variable in size, then FIFO can have very poor average response time.
- If tasks are equal in size, FIFO is optimal in terms of average response time.
- Considering only the processor, SJF is optimal in terms of average response time.
- SJF is pessimal in terms of variance in response time.
- If tasks are variable in size, Round Robin approximates SJF.
- If tasks are equal in size, Round Robin will have very poor average response time.
- Tasks that intermix processor and I/O benefit from SJF and can do poorly under Round Robin.
- Max-min fairness can improve response time for I/O-bound tasks.
- Round Robin and Max-min fairness both avoid starvation.
- By manipulating the assignment of tasks to priority queues, an MFQ scheduler can achieve a balance between responsiveness, low overhead, and fairness.

In the rest of this chapter, we extend these ideas to multiprocessors, energy-constrained environments, real-time settings, and overloaded conditions.

7.2 Multiprocessor Scheduling

Today, most general-purpose computers are multiprocessors. Physical constraints in circuit design make it easier to add computational power by adding processors, or cores, onto a single chip, rather than making individual processors faster. Many high-end desktops and